

K8s-02

1. Create a Simple Pod Using YAML Task: Write a YAML file to create a Pod named first pod with a nginx container. Verify the Pod creation using `kubectl get pods` and check the logs of the container using `kubectl logs firstpod`.

- Now open master and create `firstpod.yaml`

```
root@Master:~# cd ~/k8s-02
root@Master:~/k8s-02# vi firstpod.yaml
```

- Now write a script for firstpod

```
apiVersion: v1
kind: Pod
metadata:
  name: firstpod
spec:
  containers:
  - name: nginx
    image: nginx
```

- Pod created and checked status running or not

```
root@Master:~/k8s-02# kubectl apply -f firstpod.yaml
pod/firstpod created
root@Master:~/k8s-02# kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
firstpod	1/1	Running	0	17s	192.168.219.73	master	<none>	<none>
testpod	1/1	Running	0	11m	192.168.219.72	master	<none>	<none>

- Check logs firstpod and get pods -o wide

```
root@Master:~/k8s-02# kubectl logs firstpod
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/06/24 16:21:36 [notice] 1#1: using the "epoll" event method
2026/06/24 16:21:36 [notice] 1#1: nginx/1.31.2
2026/06/24 16:21:36 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/06/24 16:21:36 [notice] 1#1: OS: Linux 7.0.0-1006-aws
2026/06/24 16:21:36 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
2026/06/24 16:21:36 [notice] 1#1: start worker processes
2026/06/24 16:21:36 [notice] 1#1: start worker process 29
2026/06/24 16:21:36 [notice] 1#1: start worker process 30
root@Master:~/k8s-02# kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
firstpod	1/1	Running	0	39s	192.168.219.73	master	<none>	<none>
testpod	1/1	Running	0	11m	192.168.219.72	master	<none>	<none>

```
root@Master:~/k8s-02#
```

2. **Set Environment Variables in a Pod Task: Modify the YAML file to include environment variables myname: sabair and City: Hyderabad. Deploy the Pod and use kubectl exec <pod_name> -- env to check if the environment variables are set properly.**

- Create Environmental variables file

```
root@Master:~/k8s-02# vi envpod.yaml
root@Master:~/k8s-02#
```

- Now write a script for environmental variables

```
apiVersion: v1
kind: Pod
metadata:
  name: envpod
spec:
  containers:
  - name: nginx
    image: nginx
    env:
    - name: myname
      value: "sabair"
    - name: City
      value: "Hyderabad"
```

- Apply environmental pod and verify pods

```
root@Master:~/k8s-02# kubectl apply -f envpod.yaml
pod/envpod created
root@Master:~/k8s-02# kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
envpod        1/1     Running   0           9s
firstpod      1/1     Running   0          10m
testpod       1/1     Running   0          21m
```

- Verify environmental variables

```
root@Master:~/k8s-02# kubectl exec envpod -- env | grep -E 'myname|City'
myname=sabair
City=Hyderabad
root@Master:~/k8s-02# kubectl exec envpod -- env | grep -E 'myname|City'
```

3. **Deploy a Pod with Commands (Args) in YAML Task: Modify the YAML file to add args that instruct the container to sleep for 50 seconds. Deploy the Pod and use kubectl describe pod to verify the args are correctly passed to the container.**

- Create Args file and write a script

```
root@Master:~/k8s-02# vi argspod.yaml
root@Master:~/k8s-02# kubectl apply -f argspod.yaml
pod/argspod created
root@Master:~/k8s-02#
```

```

apiVersion: v1
kind: Pod
metadata:
  name: argspod
spec:
  containers:
  - name: sleeper
    image: busybox
    args:
    - sleep
    - "50"

```

- File created and verify pods

```

root@Master:~/k8s-02# kubectl apply -f argspod.yaml
pod/argspod created
root@Master:~/k8s-02# kubectl get pods

```

NAME	READY	STATUS	RESTARTS	AGE
argspod	1/1	Running	1 (20s ago)	71s
envpod	1/1	Running	0	7m42s
firstpod	1/1	Running	0	18m
testpod	1/1	Running	0	29m

- Verify argspod

```

root@Master:~/k8s-02# kubectl describe pod argspod | grep -A2 Args
    Args:
        sleep
        50
root@Master:~/k8s-02# kubectl describe pod argspod | grep -A2 Args

```

4. **Create a Pod with Two Containers Task:** Create a YAML file to define a Pod with two nginx containers inside. Use kubectl exec to access both containers and verify that both containers can communicate through the same network (e.g., using telnet between them).

- Create multi containers file and write script

```

root@Master:~/k8s-02# vi multicontainer.yaml
root@Master:~/k8s-02# 

```

```

apiVersion: v1
kind: Pod
metadata:
  name: multicontainer
spec:
  containers:
  - name: nginx
    image: nginx

  - name: busybox
    image: busybox
    command:
    - sh
    - -c
    - sleep 360

```

- Apply multi container and get pods

```
root@Master:~/k8s-02# kubectl apply -f multicontainer.yaml
pod/multicontainer created
root@Master:~/k8s-02# kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
argspod	0/1	CrashLoopBackOff	6 (67s ago)	12m
envpod	1/1	Running	0	19m
firstpod	1/1	Running	0	29m
multicontainer	2/2	Running	0	8s
testpod	1/1	Running	0	40m

```
root@Master:~/k8s-02# 
root@Master:~/k8s-02# kubectl exec -it multicontainer -c nginx -- sh
# exit
root@Master:~/k8s-02# kubectl exec -it multicontainer -c busybox -- sh
/ # exit
```

5. Set Up an Init Container in a Pod Task: Modify the YAML to include an init container that sleeps for 30 seconds before the main containers start. Verify the init container's execution using kubectl describe pod and check the logs to confirm its completion.

- Create Up an Init container and write script

```
root@Master:~/k8s-02# vi initpod.yaml
root@Master:~/k8s-02#
```

```
apiVersion: v1
kind: Pod
metadata:
  name: initpod
spec:
  initContainers:
  - name: init-container
    image: busybox
    command:
    - sh
    - -c
    - sleep 30
  containers:
  - name: nginx
    image: nginx
```

- Now apply init pod and get pods

```
root@Master:~/k8s-02# kubectl apply -f initpod.yaml
pod/initpod created
root@Master:~/k8s-02# kubectl get pods -w
```

NAME	READY	STATUS	RESTARTS	AGE
envpod	1/1	Running	0	27m
firstpod	1/1	Running	0	37m
initpod	0/1	Init:0/1	0	8s
multicontainer	2/2	Running	0	8m13s
testpod	1/1	Running	0	48m
initpod	0/1	PodInitializing	0	31s
initpod	1/1	Running	0	32s

- This proves the init container ran first, slept for 30 seconds, completed successfully, and only then the nginx container started.

```
root@Master:~/k8s-02# kubectl logs initpod -c init-container
root@Master:~/k8s-02# kubectl describe pod initpod | grep -A5 "Init Containers"
Init Containers:
  init-container:
    Container ID:   containerd://df52a9b5cf0f7f70c4086cbb772a4aebc8818f0ebe024e8376674d9da7d5e3a6
    Image:          busybox
    Image ID:       docker.io/library/busybox@sha256:fd8d9aa63ba2f0982b5304e1ee8d3b90a210bc1ffb5314d980eb6962f1a9715d
    Port:          <none>
```

6. **Run a Dry Run Command to Generate YAML Task:** Use the `kubectl run nginx --image=nginx --dry-run=client -o yaml` command to generate a Pod YAML definition. Modify the generated YAML to suit specific requirements (e.g., labels or environment variables) and deploy it.

```
root@Master:~/k8s-02# kubectl run nginx --image=nginx --dry-run=client -o yaml > nginx.yaml
root@Master:~/k8s-02# cat nginx.yaml
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: nginx
  name: nginx
spec:
  containers:
  - image: nginx
    name: nginx
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Always
status: {}
```

```
root@Master:~/k8s-02# vi nginx.yaml
root@Master:~/k8s-02# vi nginx.yaml
root@Master:~/k8s-02#
```

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    app: nginx
    env: dev
    name: nginx
spec:
  containers:
  - image: nginx
    name: nginx
    env:
    - name: MY_ENV
      value: DEV
```

```

root@Master:~/k8s-02# vi nginx.yaml
root@Master:~/k8s-02# kubectl apply -f nginx.yaml
pod/nginx created
root@Master:~/k8s-02# kubectl get pods

```

NAME	READY	STATUS	RESTARTS	AGE
envpod	1/1	Running	0	41m
firstpod	1/1	Running	0	51m
initpod	1/1	Running	0	14m
multicontainer	2/2	Running	0	22m
nginx	1/1	Running	0	8s
testpod	1/1	Running	0	62m

```

root@Master:~/k8s-02# kubectl get pod nginx --show-labels
NAME      READY   STATUS    RESTARTS   AGE   LABELS
nginx     1/1     Running   0           18s   app=nginx,env=dev
root@Master:~/k8s-02# kubectl exec nginx -- env | grep MY_ENV
MY_ENV=DEV
root@Master:~/k8s-02#

```

7. Use kubectl apply vs kubectl create Task: Create a YAML file to define a Pod. First, deploy it using kubectl create -f <file_name>.yaml and then modify the YAML (e.g., change the image version). Use kubectl apply to redeploy and verify the difference between both commands.

- Create a new file write a script

```

root@Master:~/k8s-02# vi createpod.yaml
root@Master:~/k8s-02#

```

```

apiVersion: v1
kind: Pod
metadata:
  name: createpod
spec:
  containers:
  - name: nginx
    image: nginx:1.25
~

```

- Create Pod using command and Verify pods

```

root@Master:~/k8s-02# kubectl create -f createpod.yaml
pod/createpod created
root@Master:~/k8s-02# kubectl get pods

```

NAME	READY	STATUS	RESTARTS	AGE
createpod	1/1	Running	0	6s
envpod	1/1	Running	0	45m
firstpod	1/1	Running	0	56m
initpod	1/1	Running	0	18m
multicontainer	2/2	Running	0	26m
nginx	1/1	Running	0	4m45s
testpod	1/1	Running	0	67m

```

root@Master:~/k8s-02#

```

- Now modify YAML and change image nginx:1.25 to nginx:latest and save it

```
apiVersion: v1
kind: Pod
metadata:
  name: createpod
spec:
  containers:
  - name: nginx
    image: nginx:latest
```

- Now Apply Changes we get an error so correct way to do delete old recreate

```
root@Master:~/k8s-02# vi createpod.yaml
root@Master:~/k8s-02# kubectl apply -f createpod.yaml
Warning: resource pods/createpod is missing the kubectl.kubernetes.io/last-applied-configuration annotation. It will be used on resources created declaratively by either kubectl create --save-config or kubectl apply.
pod/createpod configured
```

- Verify Image

```
root@Master:~/k8s-02# kubectl create -f createpod.yaml
Error from server (AlreadyExists): error when creating "createpod.yaml": pods "createpod" already exists
root@Master:~/k8s-02# kubectl describe pod createpod | grep Image:
Image:          nginx:latest
root@Master:~/k8s-02#
```

- Edit an Existing Pod Configuration Task: Use kubectl edit pod <pod_name> to modify the running Pod's environment variables or image. After making the changes, verify if they took effect by checking the container logs or environment variables using kubectl exec.**

- Open exist pod envpod

```
root@Master:~/k8s-02# kubectl edit pod envpod
error: pods "envpod" is invalid
```

- Open and search image: nginx

```
apiVersion: v1
kind: Pod
metadata:
  annotations:
    cnf.projectcalico.org/containerID: 891fa8329a74d99bae0
    cnf.projectcalico.org/podIP: 192.168.219.85/32
    cnf.projectcalico.org/podIPs: 192.168.219.85/32
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"Pod","metadata":{"annotations":{"name":"City","value":"Hyderabad"}},"image":"nginx","
  creationTimestamp: "2026-06-24T16:32:03Z"
  name: envpod
  namespace: default
  resourceVersion: "15463"
  uid: 49488eed-a503-4c90-acbf-3ed8d0e781c1
spec:
  containers:
  - env:
    - name: myname
      value: sabair
    - name: City
      value: Hyderabad
    image: nginx
    imagePullPolicy: Always
    name: nginx
    resources: {}
    terminationMessagePath: /dev/termination-log
-- VISUAL --
```

- Edit image nginx to nginx: latest and save

```
uid: 49488eed-a503-4c90-acbf-3ed8d0e781c1
spec:
  containers:
  - env:
    - name: myname
      value: sabair
    - name: City
      value: Hyderabad
    image: nginx:latest
    imagePullPolicy: Always
    name: nginx
    resources: {}
    terminationMessagePath: /dev/termination-log
-- INSERT --
```

- It throws an error not saved

```
root@Master:~/k8s-02# kubectl edit pod envpod
error: pods "envpod" is invalid
A copy of your changes has been stored to "/tmp/kubectl-edit-3145087783.yaml"
error: Edit cancelled, no valid changes were saved.
```

- Check image and current env variables

```
root@Master:~/k8s-02# kubectl describe pod envpod | grep Image:
Image:          nginx
root@Master:~/k8s-02# kubectl exec envpod -- env | grep -E 'myname|City'
myname=sabair
City=Hyderabad
root@Master:~/k8s-02#
```

9. **Expose a Pod Using a Service Task: Create a YAML file to expose your first pod using a Service (ClusterIP). Ensure that your service is exposing the Pod on port 80 and verify it using kubectl get svc.**

- Check if firstpod has a label

```
root@Master:~/k8s-02# kubectl get pod firstpod --show-labels
NAME          READY   STATUS    RESTARTS   AGE   LABELS
firstpod      1/1     Running   1 (25m ago)  14h   <none>
```

- Update firstpod.yaml

```
root@Master:~/k8s-02# kubectl get pod firstpod --show-labels
NAME          READY   STATUS    RESTARTS   AGE   LABELS
firstpod      1/1     Running   1 (25m ago)  14h   <none>
root@Master:~/k8s-02# vi firstpod.yaml
root@Master:~/k8s-02#
```



```

apiVersion: v1
kind: Pod
metadata:
  name: firstpod
  labels:
    app: nginx
spec:
  containers:
  - name: nginx
    image: nginx

```

- Delete and create pod firstpod

```

root@Master:~/k8s-02# vi firstpod.yaml
root@Master:~/k8s-02# kubectl delete pod firstpod
pod "firstpod" deleted
root@Master:~/k8s-02# kubectl apply -f firstpod.yaml
pod/firstpod created

```

- Create the Service YAML

```

apiVersion: v1
kind: Service
metadata:
  name: firstpod-service
spec:
  selector:
    app: nginx
  ports:
  - protocol: TCP
    port: 80
    targetPort: 80
  type: ClusterIP

```

- Apply the Service

```

root@Master:~/k8s-02# vi service.yaml
root@Master:~/k8s-02# kubectl apply -f service.yaml
service/firstpod-service created
root@Master:~/k8s-02# 

```

- Verify the Service and Endpoints

```

root@Master:~/k8s-02# kubectl get svc

```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
firstpod-service	ClusterIP	10.99.160.6	<none>	80/TCP	41s
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	20h

```

root@Master:~/k8s-02# kubectl get endpoints firstpod-service

```

NAME	ENDPOINTS	AGE
firstpod-service	192.168.219.91:80,192.168.219.92:80	52s

```

root@Master:~/k8s-02# 

```

- matches **every Pod** with the label app=nginx check labels

```
root@Master:~/k8s-02# kubectl get pods --show-labels
NAME          READY   STATUS    RESTARTS   AGE   LABELS
createpod     1/1     Running   1 (34m ago)  13h   <none>
envpod        1/1     Running   1 (34m ago)  13h   <none>
firstpod      1/1     Running   0           5m8s   app=nginx
initpod       1/1     Running   1 (34m ago)  13h   <none>
multicontainer 2/2     Running   2 (34m ago)  13h   <none>
nginx         1/1     Running   1 (34m ago)  13h   app=nginx,env=dev
testpod       1/1     Running   1 (34m ago)  14h   run=testpod
root@Master:~/k8s-02#
```

10. Pod with Resource Limits and Requests Task: Add resource requests and limits to the containers in your YAML file. Specify CPU and memory requests/limits for both containers and deploy the Pod. Use kubectl describe pod to verify if the resource configurations are correctly applied

- Create a Pod with Resource file

```
root@Master:~/k8s-02# vi resourcepod.yaml
```

- Write script

```
apiVersion: v1
kind: Pod
metadata:
  name: resourcepod
spec:
  containers:
  - name: nginx1
    image: nginx
    resources:
      requests:
        cpu: "250m"
        memory: "128Mi"
      limits:
        cpu: "500m"
        memory: "256Mi"
  - name: busybox
    image: busybox
    command:
    - sh
    - -c
    - sleep 3600
    resources:
      requests:
        cpu: "250m"
        memory: "128Mi"
      limits:
        cpu: "500m"
        memory: "256Mi"
```

- Apply it and check verify pods status

```

root@Master:~/k8s-02# vi resourcepod.yaml
root@Master:~/k8s-02# kubectl apply -f resourcepod.yaml
pod/resourcepod created
root@Master:~/k8s-02# kubectl get pods

```

NAME	READY	STATUS	RESTARTS	AGE
createpod	1/1	Running	1 (37m ago)	13h
envpod	1/1	Running	1 (37m ago)	14h
firstpod	1/1	Running	0	8m30s
initpod	1/1	Running	1 (37m ago)	13h
multicontainer	2/2	Running	2 (37m ago)	13h
nginx	1/1	Running	1 (37m ago)	13h
resourcepod	2/2	Running	0	6s
testpod	1/1	Running	1 (37m ago)	14h

- Check resource configuration

```

root@Master:~/k8s-02# kubectl describe pod resourcepod | grep -A12 "Limits"
Limits:
  cpu: 500m
  memory: 256Mi
Requests:
  cpu: 250m
  memory: 128Mi
Environment: <none>
Mounts:
  /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-hv782 (ro)
busybox:
  Container ID: containerd://4f64867e7b665be41b35e76c844b1b09c831b3cc8ead064cc6c8f8efee511b23
  Image: busybox
  Image ID: docker.io/library/busybox@sha256:fd8d9aa63ba2f0982b5304e1ee8d3b90a210bc1ffb5314d980eb6962f1a9715d
Limits:
  cpu: 500m
  memory: 256Mi
Requests:
  cpu: 250m
  memory: 128Mi
Environment: <none>
Mounts:
  /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-hv782 (ro)
Conditions:
  Type              Status
  PodReadyToStartContainers  True
  Initialized        True
root@Master:~/k8s-02#

```